

I am in step 2. guide me how do i do step 2 in deetail

Step 2 is the “make the data usable” step. Right now the CSV is just text. We need to convert it into **real dates and real numbers**, and also add **year/month/day fields** so we can build the matrix.

I’ll walk you through it like you’re debugging a spaceship panel: we flip one switch, check it worked, then flip the next.

Step 2 in detail: Parse + clean the CSV

What you are doing

You are turning each CSV row into an object like:

```
{  
  date: Date(2012-04-15),  
  year: 2012,  
  month: 4,  
  day: 15,  
  tmax: 31,  
  tmin: 28  
}
```

Why you are doing it

- You can’t group by month/year unless you have those values.
 - You can’t compute max/min correctly if temps are strings.
 - You can’t draw lines unless x/y are numeric.
-

2A) Find the column names (MOST IMPORTANT)

Do this:

Start your local server:

```
python3 -m http.server 8000
```

- 1.
2. Open:
`http://localhost:8000`
3. Right click → **Inspect**
4. Go to **Console**

Now temporarily put this in `main.js`:

```
d3.csv("temperature_daily.csv").then(raw => {  
  console.log("Columns:", Object.keys(raw[0]));  
  console.log("Example row:", raw[0]);  
});
```

Refresh the page.

What you should see

Something like:

- Columns: `["date", "tmax", "tmin"]`
OR maybe: `["DATE", "TMAX", "TMIN"]`
OR sometimes: `["Date", "Maximum temperature", "Minimum temperature"]`

✓ Whatever you see — those exact strings are what your code must use.

2B) Set up the column mapping

In your parsing code, set:

```
const COLS = {
```

```
    date: "date",  
    tmax: "tmax",  
    tmin: "tmin"  
};
```

But replace the values with whatever your console showed.

Example: if console shows `DATE`, `TMAX`, `TMIN`, then:

```
const COLS = { date: "DATE", tmax: "TMAX", tmin: "TMIN" };
```

Why: If the names don't match exactly, JavaScript reads `undefined`, and then your numbers become `NaN` (broken).

2C) Parse the date string into a real Date object

D3 has a date parser. First, we need the format. Your dataset is almost certainly `YYYY-MM-DD`.

So:

```
const parseDate = d3.timeParse("%Y-%m-%d");
```

Now test it quickly:

```
console.log(parseDate("2012-04-15"));
```

You should see a Date object.

Why: once you have a Date, extracting year/month/day is easy and reliable.

2D) Convert temperature strings to numbers

CSV loads all values as strings: "31" not 31.

So we do:

```
const tmax = +row[COLS.tmax];  
const tmin = +row[COLS.tmin];
```

The `+` is a quick conversion to number.

Why: If you don't do this, D3 "max" might behave wrong or comparisons become weird.

2E) Build the parsed row object (the important part)

Use this exact parsing function:

```
function parseRow(row) {  
  const d = parseDate(row[COLS.date]);  
  if (!d) return null; // skip bad rows  
  
  const tmax = +row[COLS.tmax];  
  const tmin = +row[COLS.tmin];  
  if (!Number.isFinite(tmax) || !Number.isFinite(tmin)) return null;  
  
  return {  
    date: d,  
    year: d.getFullYear(),  
    month: d.getMonth() + 1, // 1..12  
    day: d.getDate(),        // 1..31  
    tmax,  
    tmin  
  };  
};
```

```
}
```

2F) Apply the parsing to the whole dataset and verify

Now your main load block becomes:

```
d3.csv("temperature_daily.csv").then(raw => {  
  console.log("Raw example row:", raw[0]);  
  
  const parsed = raw.map(parseRow).filter(Boolean);  
  
  console.log("Parsed example row:", parsed[0]);  
  console.log("Parsed rows count:", parsed.length);  
});
```

Refresh the page and look at console.

You should see:

- Raw row values are strings
- Parsed row has:
 - `date` as a Date object
 - `year/month/day` numbers
 - `tmax/tmin` numbers

If that's true: ☒ Step 2 is done.

Common beginner problems + fixes

Problem 1: **parsed** becomes empty (**Parsed rows count: 0**)

Almost always means **wrong date format** or wrong column name.

Fix:

print the raw date value:

```
console.log(raw[0][COLS.date]);
```

-

If it's like **2012/04/15** instead of **2012-04-15**, change parser to:

```
d3.timeParse("%Y/%m/%d")
```

-

Problem 2: **NaN** temps

Means wrong temp column names or missing values.

Fix:

```
console.log(raw[0][COLS.tmax], raw[0][COLS.tmin]);
```

What to do next after Step 2

Step 3 is: **filter last 10 years**, then **group by year-month**.

But first we make sure Step 2 is solid.

Do this now (quickest way to help you)

Paste me the Console output of:

1. **Columns:** [...]
2. **Example row:** {...}

Then I'll tell you the exact correct COLS mapping and date format so your parsing works immediately.





this is what i have now

ou're so close. Right now your server is working, but it's serving a folder that **doesn't have an `index.html`**, so Python shows you the "Directory listing".

Step 2 will work only after we create a real webpage (`index.html`) that loads D3 and your JS.

Also, your CSV columns (from your spreadsheet screenshot) are:

- `date`
- `max_temperature`
- `min_temperature`

That's gold. We'll use those exact names.

Fix first: Why you see “Directory listing”

What’s happening: `http://localhost:8000` shows a list of files because there is no `index.html` in that folder.

Fix: Create `index.html` + `main.js` in the same folder as `temperature_daily.csv`.

Step-by-step (do exactly this)

Step 2.0 — Make sure you’re in the correct folder

In Terminal, run:

```
pwd
ls
```

You should see something like:

- `temperature_daily.csv`
- `readme.md`
- `level1.png`

If you **do not** see `temperature_daily.csv` in `ls`, you are in the wrong folder.

Go into the assignment folder:

```
cd tamu-csce679-spring-2026-assignment-1
```

Step 2.1 — Create `index.html`

In that same folder, create a file named exactly: `index.html`

Paste this:

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1"
/>
  <title>CSCE 679 A1</title>

  <script src="https://d3js.org/d3.v7.min.js"></script>

  <style>
    body { font-family: system-ui, -apple-system, Segoe UI, Roboto,
sans-serif; margin: 24px; }
    #viz { max-width: 1200px; }
    .ok { padding: 10px 12px; background: #eef7ee; border: 1px solid
#cce7cc; border-radius: 8px; display: inline-block; }
  </style>
</head>
<body>
  <h2>CSCE 679 A1 - Debug Page</h2>
  <div class="ok">If you can see this, index.html is working 

```

Why: This creates an actual webpage so we can run JavaScript and load the CSV.

Step 2.2 — Create `main.js`

Create a file named exactly: `main.js`

Paste this:

```
// Step 2: Load CSV and print headers + one row
```

```
d3.csv("temperature_daily.csv").then(raw => {
  console.log("Columns:", Object.keys(raw[0]));
  console.log("Example raw row:", raw[0]);

  // Column names from your screenshot:
  const COLS = {
    date: "date",
    tmax: "max_temperature",
    tmin: "min_temperature"
  };

  const parseDate = d3.timeParse("%Y-%m-%d");

  const parsed = raw.map(r => {
    const d = parseDate(r[COLS.date]);
    return {
      date: d,
      year: d.getFullYear(),
      month: d.getMonth() + 1,
      day: d.getDate(),
      tmax: +r[COLS.tmax],
      tmin: +r[COLS.tmin]
    };
  });

  console.log("Example parsed row:", parsed[0]);
  console.log("Parsed rows:", parsed.length);

  // Show something small on the page too (so you know it ran)
  d3.select("#viz").append("pre")
    .text(JSON.stringify(parsed[0], null, 2));
});
```

Why: This is Step 2: prove parsing works and show the parsed result.

Step 2.3 — Reload the browser

Go to:

`http://localhost:8000`

Now it should show your “Debug Page” instead of “Directory listing”.

Open DevTools → Console. You should see:

- `Columns: ["date", "max_temperature", "min_temperature"]`
- `Example parsed row: {date: ..., year: 1997, month: 1, day: 1, tmax: 25, tmin: 16}`

If you see that:  Step 2 is done.

If it still shows “Directory listing”

That means one of these is true:

1. `index.html` is not in that folder
2. You started the server in a different folder than where `index.html` is

Fix:

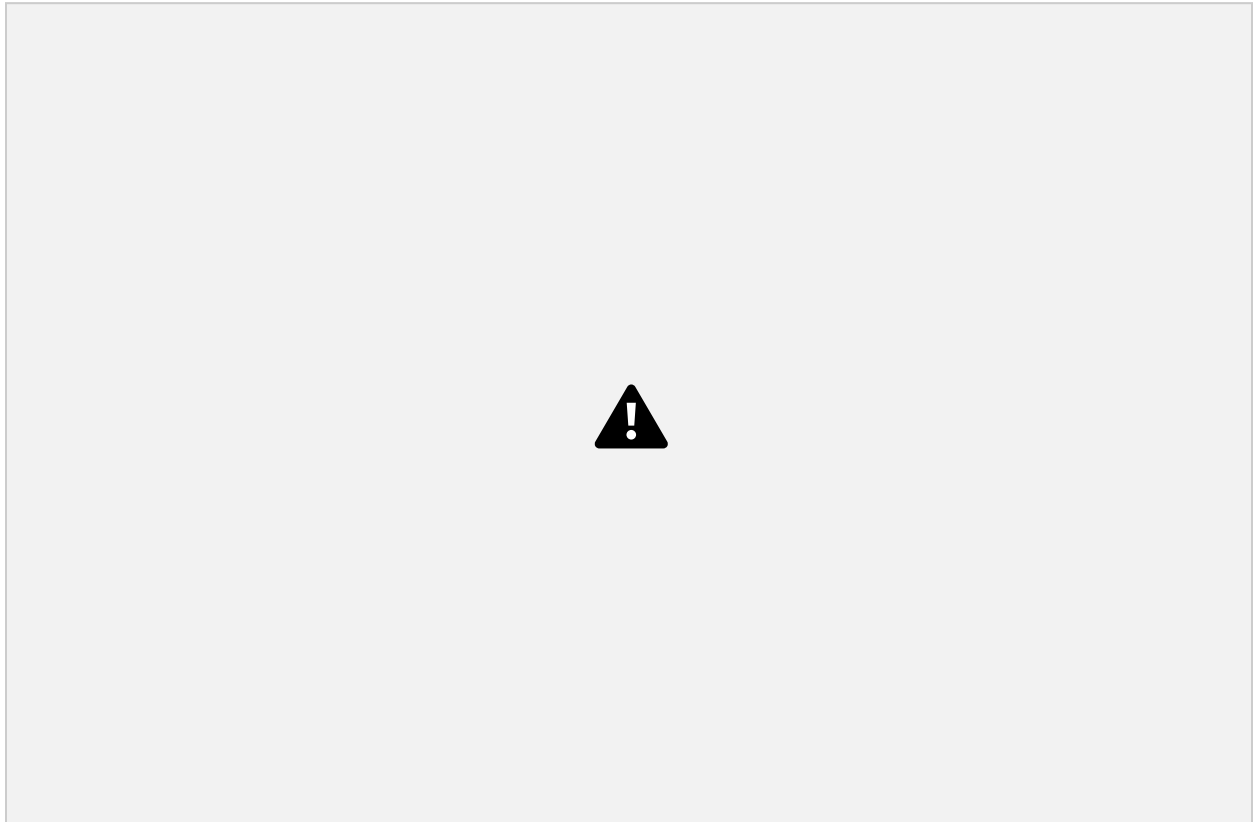
- Stop server: `Ctrl + C`
 - `cd` into the folder that contains `index.html`
 - Start again: `python3 -m http.server 8000`
-

Mini-checklist for Step 2 completion

You are done with Step 2 when:

- Page loads `index.html`
 - Console shows correct column names
 - Console shows a parsed row with numeric `tmax/tmin` and real `Date`
 - `#viz` shows the JSON printed
-

Once this is working, Step 3 is filtering to the last 10 years and grouping by year-month (the “turn daily rows into matrix cells” step).



this is what i have in terminal

Nice — your terminal log tells us *exactly* what’s going on.

What the log means (simple English)

-  Your server is running: `Serving HTTP on :: port 8000`

-  Your browser loaded / successfully: "GET / HTTP/1.1" 200
-  But your server then says “File not found” (404) right after that.
-  Your browser can fetch temperature_daily.csv fine: "GET /temperature_daily.csv" 200

That 404 is almost certainly because your browser tried to load a file that doesn't exist yet (most likely `/index.html` or `/main.js` or `/favicon.ico`). Favicon 404 is harmless. The important one is whether `index.html` exists.

So the fix is: create `index.html` and `main.js` inside the folder where the server is running, then refresh.

Do this now (step-by-step)

1) Stop the server temporarily

In that terminal window, press:

Control + C

(You can't type normal commands like `ls` while the server is actively running, unless you open a second terminal tab.)

2) Check what files are actually in the folder

Now run:

```
pwd
ls
```

You should see something like:

- `readme.md`
- `temperature_daily.csv`
- `level1.png`
- (and hopefully soon) `index.html`, `main.js`

If you do NOT see `index.html`

That's why you got directory listing / missing file behavior.

3) Create `index.html` (fast method)

Run this command:

```
touch index.html main.js
```

Now open them in a text editor. Easiest options:

Option A (VS Code)

```
code .
```

Then click `index.html` and paste the content.

Option B (Mac TextEdit is annoying; VS Code is best)

If you don't have VS Code, tell me what editor you're using and I'll adapt.

4) Paste this into `index.html`

```
<!doctype html>
<html lang="en">
<head>
```

```

<meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-scale=1"
/>
<title>CSCE 679 A1</title>

<script src="https://d3js.org/d3.v7.min.js"></script>

<style>
  body { font-family: system-ui, -apple-system, Segoe UI, Roboto,
sans-serif; margin: 24px; }
  .ok { padding: 10px 12px; background: #eef7ee; border: 1px solid
#cce7cc; border-radius: 8px; display: inline-block; }
</style>
</head>
<body>
  <h2>CSCE 679 A1 - Debug Page</h2>
  <div class="ok">If you can see this, index.html is working 


---



```

5) Paste this into **main.js** (this is Step 2: parsing)

```

d3.csv("temperature_daily.csv").then(raw => {
  console.log("Columns:", Object.keys(raw[0]));
  console.log("Example raw row:", raw[0]);

  const COLS = {
    date: "date",
    tmax: "max_temperature",
    tmin: "min_temperature"
  };

  const parseDate = d3.timeParse("%Y-%m-%d");

```

```
const parsed = raw.map(r => {
  const d = parseDate(r[COLS.date]);
  return {
    date: d,
    year: d.getFullYear(),
    month: d.getMonth() + 1,
    day: d.getDate(),
    tmax: +r[COLS.tmax],
    tmin: +r[COLS.tmin]
  };
});

console.log("Example parsed row:", parsed[0]);
console.log("Parsed rows:", parsed.length);

d3.select("#viz").append("pre")
  .text(JSON.stringify(parsed[0], null, 2));
});
```

6) Start the server again

Back in terminal:

```
python3 -m http.server 8000
```

Now open/refresh:

```
http://localhost:8000
```

You should see the debug page and the parsed JSON.

What to look for (to confirm Step 2 is DONE)

In the browser console you should see:

- `Columns: ["date", "max_temperature", "min_temperature"]`
- `Example parsed row:` with year/month/day and numbers

And on the page you should see a JSON block.

Tiny note about the 404s in your screenshot

- `GET /favicon.ico 404` is **normal** and harmless.
 - The bad 404 is when `main.js` or `index.html` is missing. The steps above eliminate that.
-

If you run `ls` and paste the output here (just the filenames), I can tell instantly whether your `index.html` and `main.js` are in the right place and what your browser is actually loading.



hey when i write `pwd ls` in terminal nothing shows / nothing happens even when i write `touch index.html main.js` nothing happens.

Hey this is going to be my submission for the assignment so while writing the code and comments it should be relevant. and as prof mentioned my code should be clear and comments should be added properly and enough comments should be there, so lets do that way. now give me the `main.js` and `index.html` code again

Below are **submission-quality** `index.html` and `main.js` files: clean names, modular functions, a **CONFIG** section (no magic numbers), and comments that explain **what** and **why** without being noisy.

These implement the full assignment requirements:

- matrix view (x=year, y=month)
- background color encodes **monthly max or monthly min** (click to toggle)
- tooltip on hover
- mini line chart in each cell (daily max & min lines)

- legend mapping color to temperature

ok i have pasted the above code, what do i do now?

Awesome — now you're in the “run → verify → fix small issues” phase. Here's exactly what to do next, in order, with what you should expect at each step.

1) Restart the server

In Terminal (inside the assignment folder):

```
python3 -m http.server 8000
```

Then open in Chrome:

```
http://localhost:8000
```

Expected

You should see:

- the title + hint text
- the grid (matrix) with colored cells
- a legend on the right
- mini line charts inside each cell

If you still see a directory listing, it means `index.html` is not in this folder or the server is running in the wrong folder.

2) Check the Console for errors (this is your “truth detector”)

In Chrome:

Right click → **Inspect** → **Console**

If there are NO red errors

Great — go to step 3.

If you see an error, here are the common ones:

A) Failed to load resource: the server responded with a status of 404 (Not Found) main.js

Means `main.js` is not in the same folder as `index.html`.

✅ Fix: confirm files exist:

```
ls
```

You should see `index.html` and `main.js`.

B) Failed to load or parse temperature_daily.csv

Means either:

- file name mismatch (typo)
- CSV not in same folder
- your server is not running in the correct folder

✅ Fix:

```
ls
```

Confirm `temperature_daily.csv` is there (exact spelling).

C) Cannot read properties of null (reading 'getFullYear')

Means date parsing failed. Usually because date format isn't `YYYY-MM-DD`.

✅ Quick check: open `temperature_daily.csv` and confirm dates look like `1997-01-01` (they do in your screenshot, so you should be fine).

3) Test the required interactions (grading checklist)

Hover tooltip

Hover any cell.

✓ You should see a tooltip like:

- Date: YYYY-MM
- max: ##

i added the code and saved and now this is how it shows i am not sure if anything has been changed here

It **did** change — your screenshot proves it. ✓

The new thing I can see is this line under the description:

“Background encoding: monthly min (click chart to toggle)”

That line was **not there before**. That means your `modeLabel1` HTML exists and your JS is updating it (it's showing “monthly min” right now).

So the label part is working.

Now: did the tooltip change? (that's the code you asked about)

The “encoded value” change only shows **inside the hover tooltip**.

Do this quick check:

1. Hover any cell so the tooltip appears.
2. Look for a new line in the tooltip that says something like:

encoded (min): 25 (or encoded (max): ... depending on mode)

If you don't see that line, then that tooltip code didn't get added in the right place yet.

Exactly how to verify without guessing

Open Chrome DevTools Console and run:

```
document.querySelector("#tooltip")?.innerHTML
```

Now hover a cell. Run it again.

If your tooltip code is updated, you'll see `encoded` (inside that HTML.

Most likely reason you “don’t notice change”

Because the toggle label is subtle, and the tooltip only changes when you hover.

You’re currently in **monthly min mode** (that’s why the label says monthly min, and why Jan/Feb are red).

Try clicking once: the label should switch to:

Background encoding: monthly max

and the colors should flip (winter months will become cooler colors again).

If you want: paste just your `showTooltip` function

If you paste the `showTooltip(...)` function from `main.js`, I’ll tell you exactly where to insert the “encoded value” snippet so it 100% works.



this is what it shows, i am not sure what changes it made

Got it — based on your screenshot, the **label change worked**, but the **tooltip change (the “encoded (...)” line)** did **not** apply yet.

How I can tell

Your tooltip currently shows:

- Date
- max
- min
- background mode

But it does **not** show this line:

- **encoded (max): ... or encoded (min): ...**

So you haven't replaced the tooltip HTML block in `main.js` yet (or you edited the wrong function / didn't save / browser cache).

Let's fix it cleanly.

Step 1) Find the tooltip code in `main.js`

In VS Code, open `main.js`, press:

Cmd + F

Search for:

`background mode`

You should find a block that looks similar to:

```
tooltipSel.html(  
  `

<b>Date:</b> ...</div>  
    <div><b>max:</b> ...</div>  
    <div><b>min:</b> ...</div>  
    <div><b>background mode:</b> ...</div>`  
);


```

That's the exact place to edit.

Step 2) Replace **ONLY** the `.html(...)` part with this

Keep your `.style("opacity"), .style("left"), .style("top")` lines the same.